

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

Search the Help Center /

Transaction lifecycle

Updated 5 months ago

Overview

A transaction is a transfer of value from a source address to a destination address on the blockchain.

There are two main types of transactions: outgoing and incoming. An outgoing transaction occurs when you initiate a transaction, either manually from the Fireblocks Console or programmatically via the API. An incoming transaction is initiated by a third party sending assets to your workspace.

A transaction's status describes its current state in the transaction workflow. Transaction sub-statuses offer more information about why a transaction may be in its current state.

Incoming and outgoing transactions have different possible statuses.

Viewing transaction statuses and substatuses

Transaction statuses and sub-statuses are displayed in different ways throughout the Fireblocks Console and API.

Fireblocks Console

- The [Recent activity panel](#) displays primary transaction statuses. Hover over the status to view sub-statuses when available.
- The [Transaction History page](#) includes all status details. You can export the [transaction log](#) to save all status details.

Transaction statuses in the Console appear in four colors representing four main stages:

- **Yellow:** Transactions that are currently processing within the Fireblocks ecosystem.
- **Blue:** Transactions that are currently processing outside Fireblocks, including all third-party activity and all blockchain-specific activity.
- **Green:** Successfully completed transactions.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C



ID, or [Get a specific transaction by external transaction ID](#) endpoints to retrieve the transaction's status.

- Use [webhook notifications](#) to receive notifications for all transaction status changes.

Transaction statuses and sub-statuses appear as their [status code](#) in the Fireblocks API and webhook notifications.

Transaction operations depending on their status

Locate the transaction card in the recent activity panel of the Fireblocks Console to see the operations available. Depending on the status and other transaction details the following actions may be available:

- [Cancel a transaction](#)
- [Retry a transaction](#)
- [Boost or drop an EVM transaction](#)
- [Boost a UTXO transaction](#)
- [Rescreen or bypass AML policy results](#)
- [Dismiss a transaction card](#)

Transaction flowchart

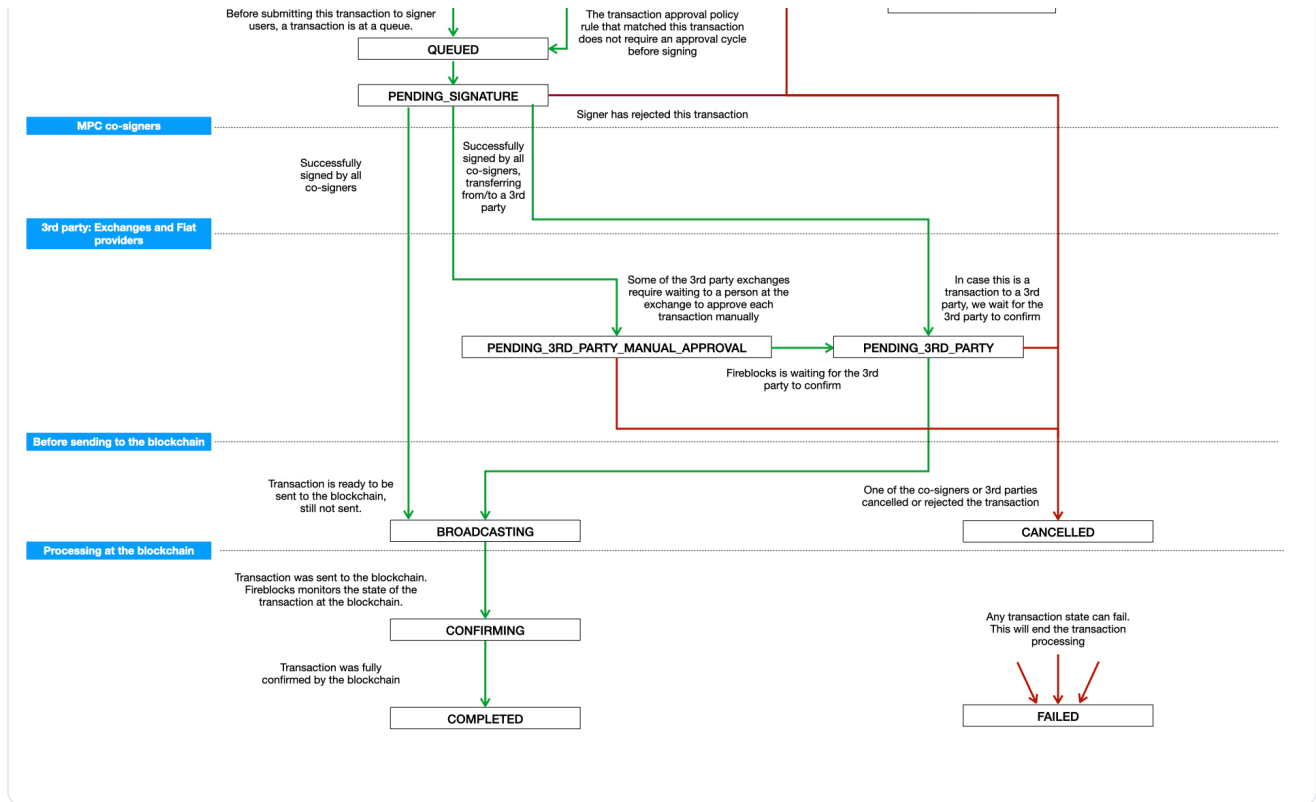
The following flowcharts show how an outgoing or incoming transaction may arrive at any state.

Outgoing transactions

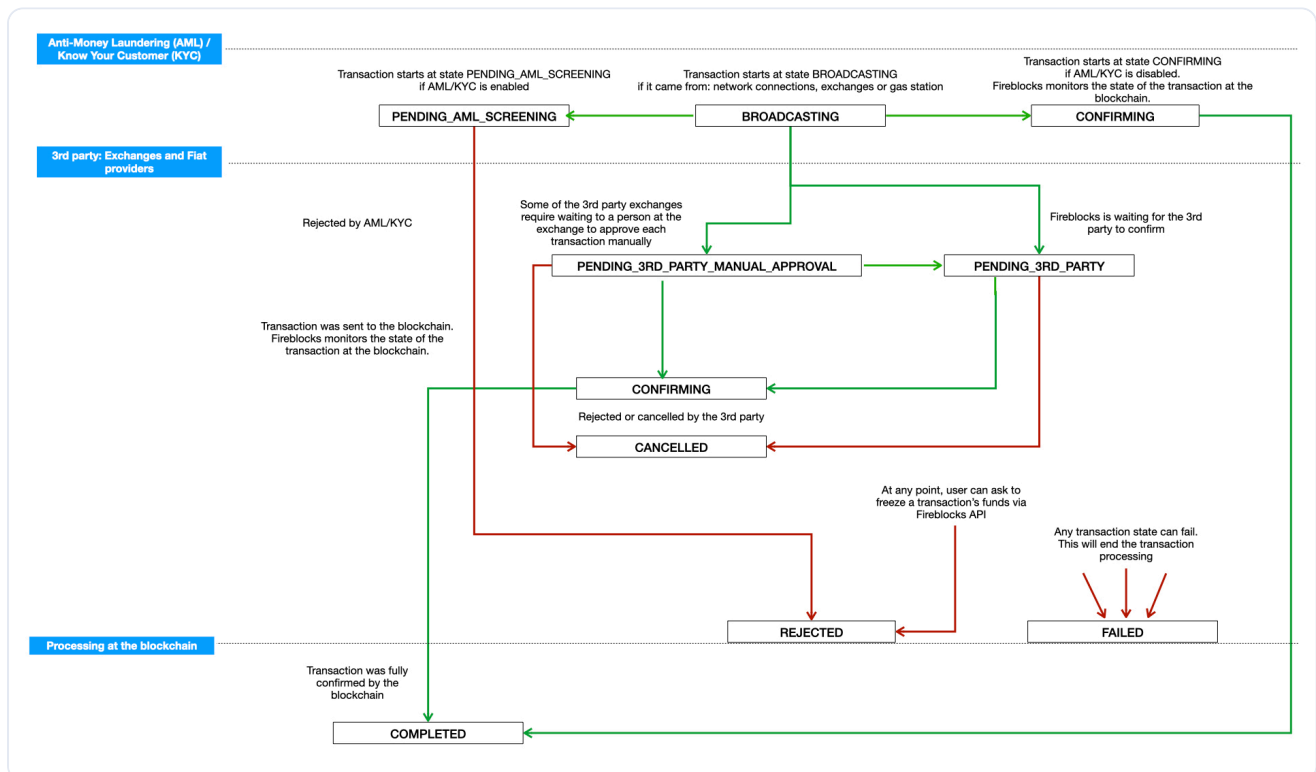
Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/



Incoming transactions



Special cases

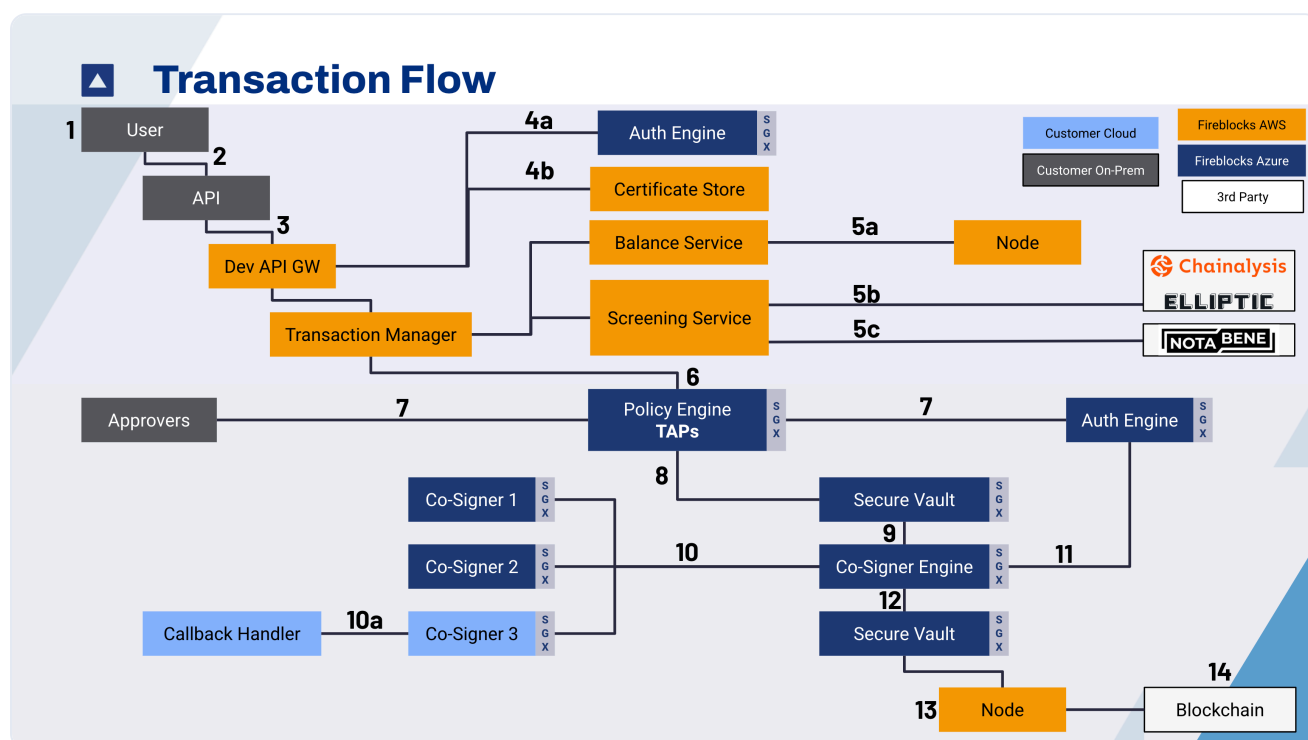
Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

/

Transaction system schematic

The following is a technical description of how transactions are processed within Fireblocks.



1. The customer creates a new transaction request through the Fireblocks Console or the Fireblocks REST API. The API performs input validation.
2. The transaction is translated to an API call with a signed JWT access token.
3. The signed request is sent to the Fireblocks Developer API Gateway.
4. The user is validated using a signed JWT access token:
 - a. The JWT signature is validated using a certificate store hosted by Fireblocks' frontend services and JWT claims. The user's CRT file is hosted [here](#).
 - b. The user's access token is validated through the Auth Engine.
5. Once validated, the request is sent for further processing by the Transaction Manager.
 - a. The Balance Microservice gets the vault balance from the node infrastructure. An insufficient balance leads to transaction failure.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

6. After the balance, AML, and Travel Rule checks run successfully, the transaction is forwarded to the [Policy Engine](#).
7. The request is sent to the Policy Engine for validation.
 - The Policies can be configured to automatically approve or require explicit approvals before the signing process begins. If the Policy rule triggered by the transaction requires approval, the approvers are notified.
 - For natively supported transaction types, the transaction details are provided when an approval request is made. Raw Signing/Contract Calls will have limited information available.
 - Each approval response is signed by the approval mobile device and validated by the Auth Service.
8. Upon receiving sufficient approvals, the signing process is initiated by a Secure Vault. PKI is built into the enclave of the Policy Engine. All services in the Fireblocks core infrastructure operate in a zero-trust configuration. Each service has a derivation of root CA and validates every handoff between services.
9. The Vault caches the transaction request and forwards the signing request to the Co-Signer Engine based on UUID and Policy rules.
10. The Co-Signer Engine requests that the co-signers validate the request within their secure enclaves using their key shards. Each co-signer validates signatures internally within the co-signer SGX enclave. Every SGX service exists in a zero-trust configuration, and data is not passed between them during the signing. Once confirmed, the partial signature hash validation is returned.
 - a. Optional: When configured, the transaction request is sent to an attached Co-Signer Callback Handler for approval.
11. The Co-Signer Engines validate the hash of the partial signatures using the Auth Engine.
12. The transaction is sent to a secure Vault, and the signed transaction is created.
13. The signed transaction is sent to the node infrastructure.
14. The transaction is submitted to the blockchain.

Transaction signing flow

The following video describes a technical overview of how transactions are signed in Fireblocks.

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help C

/

0:00 / 1:00

User participation in transaction flows

Note

Some operations are bound to specific user roles. For more information, see [User roles](#).

- **Initiator:** The user that initiated the transaction must have Initiate Transaction permissions granted.
- **Approvers:** One or more users receive the transaction for approval. For each transaction, the approvers are selected according to the [Policy](#). By default, a specified number of users is required to approve each transaction. If at least one person chooses to reject a transaction, the transaction is rejected.
- **Signer:** One designated user is selected for cryptographically signing the transaction. For each transaction, the designated signer is selected according to the [Policy](#). Fireblocks uses multi-party computation (MPC) to ensure no single entity has the full private key at any point. Using MPC technology along with reliance on SGX servers, secure enclaves, and the Policy Engine, Fireblocks provides multi-layer protection against cyberattacks.
- **Third parties:** Third parties can be either exchanges or fiat providers. Transactions that include third parties will wait for confirmation from the third parties involved.

Don't see what you're looking for? Log in for the full Help Center experience

Was this article helpful?

Yes

No

ON THIS PAGE

- Overview

Viewing transaction statuses and substatuses

Transaction operations depending on their status

Transaction flowchart

Transaction system schematic

Transaction signing flow

User participation in transaction flows

Related Articles

- 

Cancel a transaction
- 

Primary transaction statuses
- 

Transactions not broadcast by Fireblocks